UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

GAME ZONE

A MINI PROJECT(25CAP-652) REPORT SUBMITTED IN PARTIAL FULFILLMENT FOR THE AWARD OF

THE DEGREE OF MASTERS

IN

MASTER OF COMPUTER APPLICATION

Submitted by

RAJIV KUMAR

25MCA20314

MASTER OF COMPUTER APPLICATION

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY

MOHALI, PUNJAB-140301

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

TABLE OF CONTENTS

Abstract 1

CHAPTER 1. INTRODUCTION 2

1.1 Project Overview 2

1.2 Objective... 2

1.3 Scope... 2

1.4 Tools Used 2

CHAPTER 2. SYSTEM DESIGN **3**

2.1 System Architecture... 3

2.2 System Flow 3

2.3 Data Flow Diagram..... 3

2.4 Input Output Design 3

2.5 User Interface Design 4

CHAPTER 3. IMPLEMENTATION **5**

3.1 Implementation Approach... 6

3.2 Technologies Used... 7

3.3 Database Structure... 8

3.4 Code Implementation 9

CHAPTER 4. TESTING AND OUTPUT **10**

4.1 Objective of Testing 10

4.2 Types of Testing 11

4.3 Test Results 12

4.4 Output Screens 14

CHAPTER 5. CONCLUSION AND FUTURE WORK... 16

5.1 Conclusion... 16

5.2 Future Work 16

REFERENCES 16

CHAPTER 6. GITHUB **17**

ABSTRACT

The *Game Zone* is a desktop-based application developed using C++ and the Win32 API that integrates multiple mini-games into a single interactive platform. The primary objective of this project is to demonstrate fundamental programming concepts such as event-driven architecture, modular design, and graphical user interface development in a simple and engaging manner.

The application consists of four main modules: Guess the Number, Rock-Paper-Scissors, Tic-Tac-Toe, and Quick Math Challenge. Each module is designed to provide an interactive experience while showcasing different logical and algorithmic concepts. The system manages user input, game states, and real-time feedback efficiently through structured programming techniques.

The user interface is built using the Win32 API, utilizing controls such as buttons, text fields, and labels to ensure smooth interaction. The application follows an event-driven approach where user actions trigger specific responses, enabling seamless navigation between different game modes.

Overall, the project serves as a practical implementation of C++ concepts by combining logic building, user interaction, and GUI design into a unified system. It provides a clear understanding of how desktop applications can be developed using low-level programming techniques while maintaining performance and usability.

In addition to entertainment, the application focuses on enhancing logical thinking and problem-solving skills. Each game module is designed in such a way that it encourages users to make decisions, analyze outcomes, and improve performance through repeated interaction. For example, the Tic-Tac-Toe module incorporates a basic computer strategy, allowing users to experience simple artificial intelligence behavior, while the Quick Math Challenge improves speed and accuracy under time constraints.

The system is developed with a modular approach, where each game operates independently while sharing a common interface. This design improves code readability, maintainability, and scalability, making it easier to add new features or additional games in the future. The use of structured programming techniques ensures that the application remains efficient and easy to understand, even for beginners.

.CHAPTER 1

INTRODUCTION

The *Game Zone* is an interactive desktop application developed using **C++ and the Win32 API** to provide a collection of mini-games within a single platform. The project focuses on implementing fundamental programming concepts such as event-driven programming, modular design, and graphical user interface development in an intuitive and user-friendly manner.

- The application provides a centralized interface where users can navigate between multiple game modules, including Guess the Number, Rock-Paper-Scissors, Tic-Tac-Toe, and Quick Math Challenge. Each module is designed to deliver an engaging experience while demonstrating different logical and computational techniques.

- In the Guess the Number module, the system generates a random number within a defined range, and the user attempts to identify it within limited attempts. The application provides real-time feedback such as "higher" or "lower," helping users refine their guesses and understand decision-making logic.

- The Rock-Paper-Scissors module allows the user to compete against the computer. The system generates random moves and determines the winner based on predefined rules. This module demonstrates the use of conditional statements, randomization, and score tracking mechanisms.

- The Tic-Tac-Toe module provides a grid-based game where the user plays against a computer-controlled opponent. The system checks for winning conditions and implements a basic strategy for the computer's moves, allowing users to understand game logic, state management, and simple AI techniques.

- The Quick Math Challenge module presents arithmetic problems to the user within a time limit. The user must solve as many problems as possible before the timer ends. This module demonstrates time-based execution, dynamic problem generation, and performance tracking.

- The graphical interface is designed using the Win32 API and follows an event-driven approach. It includes buttons, text fields, and labels to facilitate user interaction. Users can easily switch between different games through a menu-driven interface.

- Overall, the project serves as both an entertainment and educational tool by combining logic, interactivity, and graphical design. It helps in strengthening programming fundamentals while providing an engaging user experience.

CHAPTER 2

System Architecture

The Game Zone application employs a modular architecture built with C++ and the Win32 API. The system is divided into the following layers:

Presentation Layer

This layer manages the graphical user interface (GUI). It is implemented using Tkinter and offers multiple interactive modules such as:

• Expression evaluator interface

• Grid-based pathfinding environment

• Graph creation and traversal panel

Users can:

• Enter expressions

• Place start and end nodes

• Draw or erase obstacles

• Add nodes and edges in graphs

• Select algorithms

• Visualize execution step-by-step

Application Layer

This layer contains the core system logic. It manages:

- Expression processing (converting infix to postfix and evaluation)
- Pathfinding algorithms like Dijkstra and Bellman-Ford
- Graph traversal algorithms like BFS and DFS
- Flow control and step-by-step visualization
- User interaction (mouse clicks, dragging, input events)
- Performance monitoring, including steps and execution flow

Data Layer

The project uses in-memory data structures instead of a database:

• Stack and list structures for expression evaluation

2D grid representation with Node objects
Adjacency list for graph modeling
Queues, stacks, and priority queues for algorithm execution

System Flow

The system operates through different modules:

Expression Module Flow:

User inputs expression → Tokenization → Conversion to postfix → Evaluation → Result display

Pathfinding Module Flow:

User configures grid → Sets start and end nodes → Draws obstacles → Selects an algorithm → Executes algorithm → Visualizes path → Displays steps

Graph Module Flow:

User adds nodes → Connects edges → Chooses traversal method → Runs algorithm → Highlights nodes step-by-step

Data Flow Diagram

Input → Processing → Output

Input: User inputs (expression, clicks, algorithm choice)

• Processing: Algorithm execution and computation

• Output: Visual displays, evaluation results, traversal paths

User Interface Design

The UI is built with Tkinter, emphasizing simplicity, interactivity, and clarity.

The application uses a tab-based layout:

• Expression evaluation tab

• Graph traversal tab

The interface includes:

• Input fields for expressions

• Canvas for grid and graph visualization

• Buttons, sliders, and dropdown menus for controls

CHAPTER 3

IMPLEMENTATION

Implementation Approach

The development of the *Game Zone* followed a modular and structured approach to ensure maintainability and scalability. The system was divided into key modules: Guess the Number, Rock-Paper-Scissors, Tic-Tac-Toe, Quick Math Challenge, and user interaction handler. Each module was developed and tested independently before integration.

Each game module focuses on implementing specific logic and user interaction. The Guess module is based on number prediction logic, the Rock-Paper-Scissors module uses conditional decision-making, the Tic-Tac-Toe module implements board logic and basic AI, and the Math module focuses on time-based problem solving.

The application updates the interface in real-time using the Win32 event-driven mechanism, enabling users to interact smoothly with the system. Event-handling captures user actions such as button clicks and inputs. Integration testing ensured smooth communication between all modules and provided a seamless user experience.

Technologies Used

C++: Core language for implementation

• Win32 API: GUI development and interaction handling

Standard Libraries: o <windows.h> – Used for window

creation and event handling o <random> – Used for random

number generation o <algorithm> – Used for utility operations

o <cstdio> – Used for input/output handling

Development Tools: Visual Studio / Code::Blocks / MinGW

Structure

- Guess Game Module
- Generates random number
- Accepts user input
- Compares input with secret number
- Provides feedback (higher/lower)
- Tracks number of attempts

Rock-Paper-Scissors Module

- User selects move (rock, paper, scissors)
- Computer generates random move
- Result is calculated based on rules
- Score is updated after each round
- Game continues until winner is decided

Tic-Tac-Toe Module

- Board represented using array
- User plays against computer
- Moves updated dynamically
- Checks for win or draw conditions
- Basic AI logic for computer moves

Quick Math Module

- Generates arithmetic questions
- Timer controls gameplay duration
- Accepts user answers
- Verifies correctness
- Updates score in real-time

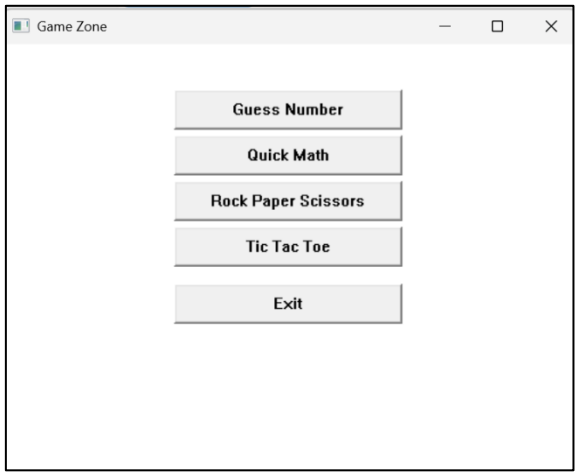
Code Implementation

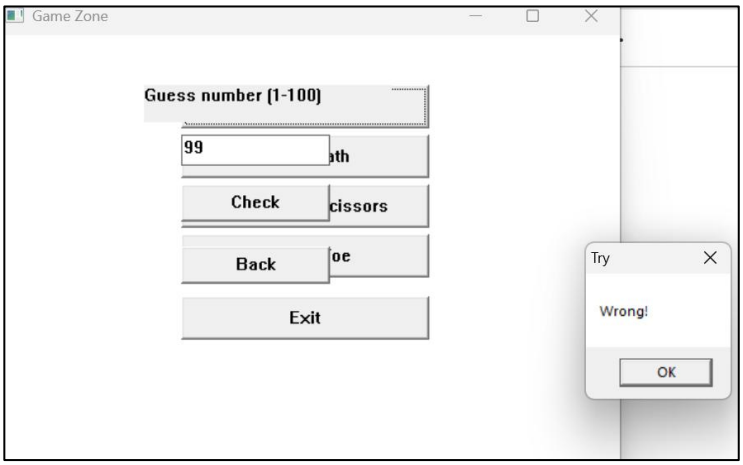
```
#include <algorithm>
#include <cstdarg>
#include <cstdio>
#include <cwchar>
#include <random> #include
<windows.h> int
cpuMoveTTT(char* board) {
auto tryWin = [&](char p) -> int {
for (int i = 0; i < 9; ++i) {            if
(board[i] != ' ')                continue;
```

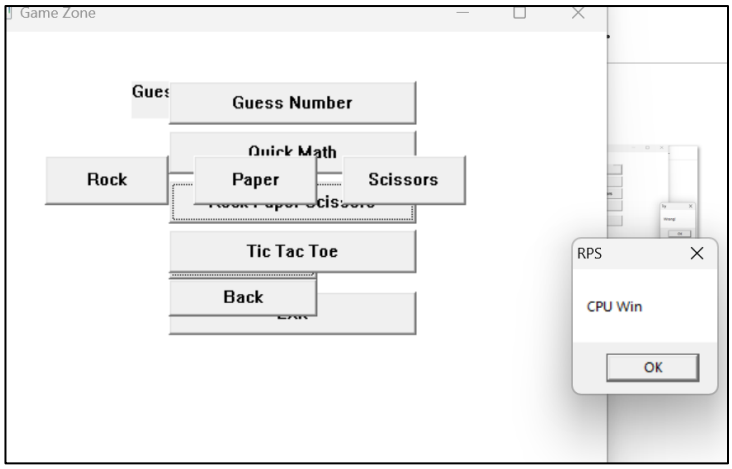
```
  board[i] = p;            bool w =
lineWinTTT(board, p);            board[i]
= ' ';            if (w)                return i;
}        return -1;
   };
   int w = tryWin('O');
if (w >= 0)
return w;     int b =
tryWin('X');     if (b
>= 0)        return b;
if (board[4] == ' ')
return 4;
   const int order[] = {0, 2, 6, 8, 1, 3, 5, 7};
   for (int i : order) {
if (board[i] == ' ')
return i;     }     return
-1;
}

void resetGuess(HWND hwnd, AppState* st) {     st-
>guessSecret = randomInt(1, 100);     st->guessTries =
0;     SetWindowTextW(GetDlgItem(hwnd,
IDC_GUESS_FEEDBACK),
                L"I'm thinking of 1–100. You have 8 tries.");
   SetWindowTextW(GetDlgItem(hwnd, IDC_GUESS_EDIT), L"");
}

void resetRPS(HWND hwnd, AppState* st) {
   st->rpsYou = st->rpsCpu = 0;
   SetWindowTextW(GetDlgItem(hwnd, IDC_RPS_SCORE), L"Score — You: 0   CPU: 0");
SetWindowTextW(GetDlgItem(hwnd, IDC_RPS_RESULT), L"First to 3 wins. Choose below."); }

void resetTTT(HWND hwnd, AppState* st) {
   st->tttOver = false;     for
(int i = 0; i < 9; ++i) {
       st->tttBoard[i] = ' ';
       HWND cell = GetDlgItem(hwnd, IDC_TTT_0 + i);
       SetWindowTextW(cell, L" ");
EnableWindow(cell, TRUE);
   }
   SetWindowTextW(GetDlgItem(hwnd, IDC_TTT_STATUS), L"You are X. Click a square.");
}
void nextMathProblem(HWND hwnd, AppState* st) {
if (!st->mathActive)        return;     const int a =
```

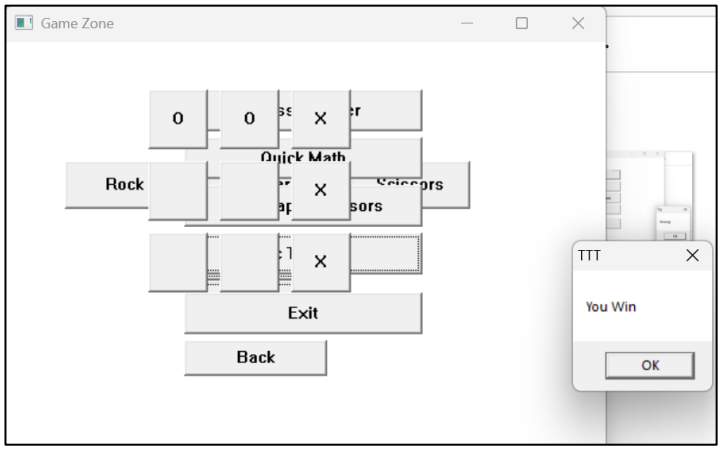
```
randomInt(2, 12), b = randomInt(2, 12);     const int op
= randomInt(0, 1);     wchar_t q[64];     if (op == 0) {
  st->mathExpected = a + b;
wfmt(q, 64, L"%d + %d = ?", a, b);
    } else {
        const int x = (std::max)(a, b), y = (std::min)(a, b);
st->mathExpected = x - y;         wfmt(q, 64, L"%d -
%d = ?", x, y);
    }
    SetWindowTextW(GetDlgItem(hwnd, IDC_MATH_PROBLEM), q);
SetWindowTextW(GetDlgItem(hwnd, IDC_MATH_EDIT), L"");
}
    case WM_TIMER: {
if (!st)            break;
        if (wParam != TIMER_MATH || !st-
>mathActive)            break;        st-
>mathSecondsLeft--;
        wchar_t t[48];
        wfmt(t, 48, L"Time left: %d s", st->mathSecondsLeft);
SetWindowTextW(GetDlgItem(hwnd, IDC_MATH_TIMER), t);
if (st->mathSecondsLeft <= 0) {
            stopMath(hwnd, st);
wchar_t done[160];
            wfmt(done, 160, L"Time's up!  Correct: %d / %d", st->mathCorrect, st->mathTotal);
SetWindowTextW(GetDlgItem(hwnd, IDC_MATH_PROBLEM), done);
        }
return 0;
    }

    default:
break;
    }
    return DefWindowProcW(hwnd, msg, wParam, lParam);
}

    MSG msg{};
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
DispatchMessageW(&msg);
    }
    return (int)msg.wParam;
```

Output


Game Zone
Guess Number
Quick Math
Rock Paper Scissors
Tic Tac Toe
Exit


Game Zone
Guess number (1-100)
99
Check
Back
Exit
Try
Wrong!
OK


Game Zone
Guess Number
Rock
Paper
Scissors
Tic Tac Toe
Back
RPS
CPU Win
OK


Game Zone
Rock
Exit
Back
TTT
You Win
OK

CHAPTER 4.

CONCLUSION

The *Game Zone* application successfully demonstrates the practical implementation of interactive games and logical challenges through an engaging and visually appealing interface. The project effectively bridges the gap between theoretical understanding of game logic and practical user interaction by allowing players to experience real-time gameplay across multiple modules such as Tic-Tac-Toe, Rock-Paper-Scissors, Guess the Number, and Quick Math Challenges.

Through the implementation of features like dynamic game boards, decision-based logic, score tracking, and timed challenges, the system highlights the working principles, rules, and behavior of each game. The realtime visualization of moves, player actions, and score updates enables users to better understand game mechanics, strategy development, and problem-solving in an interactive environment.

The use of Python and Tkinter provides a lightweight yet powerful platform for building an intuitive graphical user interface. Interactive features such as dynamic board updates, button-based inputs, animation feedback, and progress tracking significantly enhance user engagement and the overall gaming experience.

Overall, the project fulfills its objective of serving as an educational and recreational tool for students and beginners in computer science. It simplifies complex logical concepts through hands-on interaction and gamified learning. The system also provides a strong foundation for future enhancements, such as adding more advanced games, integrating AI opponents, improving visual animations, and expanding functionality to include multiplayer support or real-time leaderboards.

.

REFERENCE

Python Official Documentation –

- **Tkinter Documentation –**
- **Al Sweigart, Automate the Boring Stuff with Python, No Starch Press, 2019 • Eric Matthes, Python Crash Course, 2nd Edition, No Starch Press, 2019 • Michael H. Goldwasser, Data Structures and Algorithms in Python, Wiley, 2013 • TutorialsPoint – Python, Game Development, and Algorithm Concepts, • GeeksforGeeks – Python Game Programming, Loops, Functions, and Tkinter GUI, • Stack Overflow – Community Q&A on Python, Tkinter, and Game Development, • Real Python – Game Development Tutorials using Python, • YouTube – Python Tkinter Game Development Tutorials and Interactive GUI Demos**

6. Github